

```
/*  
JORGE MEJIAS DONOSO - ALBERTO MENCHEN MONTERO - 206  
100495807@alumons.uc3m.es 100495692@alumnos.uc3m.es  
*/
```

```
%{          // SECCION 1 Declaraciones de C-Yacc
```

```
#include <stdio.h>
```

```
#include <ctype.h>    // declaraciones para tolower
```

```
#include <string.h>   // declaraciones para cadenas
```

```
#include <stdlib.h>   // declaraciones para exit ()
```

```
#define FF fflush(stdout); // para forzar la impresion inmediata
```

```
int yylex ();
```

```
int yyerror ();
```

```
char *mi_malloc (int);
```

```
char *gen_code (char *);
```

```
char *int_to_string (int);
```

```
char *char_to_string(char);
```

```
char temp [2048];
```

```
// Abstract Syntax Tree (AST) Node Structure
```

```
typedef struct ASTnode t_node;
```

```
struct ASTnode {  
    char *op ;  
    int type ;           // leaf, unary or binary nodes  
    t_node *left ;  
    t_node *right ;  
};
```

```
// Definitions for explicit attributes
```

```
typedef struct s_attr {
```

```
int value ; // - Numeric value of a NUMBER

char *code ; // - to pass IDENTIFIER names, and other translations

t_node *node ; // - for possible future use of AST

}t_attr ;
```

```
#define YYSTYPE t_attr
```

```
%}
```

```
// Definitions for explicit attributes
```

```
%token NUMBER
```

```
%token IDENTIF // Identificador=variable
```

```
%token INTEGER // identifica el tipo entero
```

```
%token STRING
```

```
%token MAIN // identifica el comienzo del proc. main
```

```
%token WHILE // identifica el bucle main
```

```
%token LOOP
```

```
%token DO
```

%token SETQ

%token SETF

%token DEFUN // identifica la definicion de una funcion

%token PRINT

%token PRINC

%token AND

%token OR

%token NOT

%token MOD

%token E

%token MET

%token GET

%token DISTINTO

%token UNARY_SIGN

%token IF

%token PROGN

%token RETURN_FROM

%left OR

%left AND

%left '=' DISTINTO

%left '<' '>' MET GET E

%left '+' '-'

%left '*' '/' MOD

%right NOT

%right UNARY_SIGN

%% // Seccion 3 Gramatica - Semantico

// Declaracion de la gramatica

axioma: '(' sentencia ')' { printf ("%s\n", \$2.code); }

r_axioma {;}

;

```
// recursividad axioma
```

```
r_axioma:          {;}
    | axioma      {;}
    ;
```

```
// declaracion de sentencia
```

```
sentencia: SETQ IDENTIF expresion_parenthesis { sprintf(temp, "variable %s\n%s %s !", $2.code, $3.code, $2.code);
    $$code = gen_code(temp); }
    | SETF IDENTIF expresion_parenthesis { sprintf(temp, "%s %s !", $3.code, $2.code);
    $$code = gen_code(temp); }
    | DEFUN MAIN '(' ')' lista_sentencias {
    sprintf(temp, ": %s\n%s\n;", $2.code, $5.code);
    $$code = gen_code (temp) ; }
    | PRINT STRING { sprintf(temp, ".\n %s\n", $2.code);
    $$code = gen_code(temp); }
    | PRINC expresion_parenthesis { sprintf(temp, "%s .", $2.code);
    $$code = gen_code(temp); }
    | PRINC STRING      { sprintf (temp, ".\n%s\n", $2.code) ;
    $$code = gen_code (temp) ; }
```

```

| LOOP WHILE expresion_parentesis DO lista_sentencias {
    sprintf(temp, "BEGIN\n%s\nWHILE\n%s\nREPEAT", $3.code, $5.code);

    $$code = gen_code (temp) ; }

| MAIN { sprintf(temp, "%s\n", $1.code);

    $$code = gen_code(temp); }

| if      { $$code = $1.code; }

| expresion { sprintf (temp, "%s", $1.code) ;

    $$code = gen_code (temp) ; }

;

```

// declaracion de expresion_parentesis para acpetar con o sin parenteis

```

expresion_parentesis: '(' expresion ')' { sprintf (temp, "%s", $2.code) ;

    $$code = gen_code (temp) ; }

| expresion { sprintf (temp, "%s", $1.code) ;

    $$code = gen_code (temp) ; }

```

// declaracion de if

```

if: IF expresion_parentesis sentencia_if sentencia_else {

    if (strlen($4.code) <= 0) {

```

```

        sprintf(temp, "%s\nIF\n%s\nTHEN", $2.code, $3.code);
    } else {
        sprintf(temp, "%s\nIF\n%s\nELSE\n%s\nTHEN", $2.code, $3.code, $4.code);
    }
    $$code = gen_code(temp);
}
;

```

// declaracion de sentencia_if

```

sentencia_if:  '(' PROGN lista_sentencias ')' {
    sprintf (temp, "%s", $3.code);
    $$code = gen_code(temp);
}
| '(' sentencia ')' {
    sprintf (temp, "%s", $2.code);
    $$code = gen_code(temp);
}

```

// declaracion de sentencia_else


```

sentencia_else: '(' PROGN lista_sentencias ')' {
    sprintf (temp, "%s", $3.code);
    $$code = gen_code(temp);
}
| '(' sentencia ')' {
    sprintf (temp, "%s", $2.code);
    $$code = gen_code(temp);
}
| /*lambda*/ {
    $$code = gen_code("");
}
;

// declaracion de lista de sentencias
lista_sentencias: '(' sentencia ')' { sprintf (temp, "%s", $2.code) ;
    $$code = gen_code (temp) ; }
| '(' sentencia ')' lista_sentencias { sprintf (temp, "%s\n%s", $2.code, $4.code) ;
    $$code = gen_code (temp) ; }
;

```

// declaracion de expresion

```
expresion:  operando      { $$ = $1 ; }

| '+' expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s +", $2.code, $3.code) ;
                $$ .code = gen_code (temp) ; }

| '-' expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s -", $2.code, $3.code) ;
                $$ .code = gen_code (temp) ; }

| '*' expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s *", $2.code, $3.code) ;
                $$ .code = gen_code (temp) ; }

| '/' expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s /", $2.code, $3.code) ;
                $$ .code = gen_code (temp) ; }

| MOD expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s mod", $2.code, $3.code) ;
                $$ .code = gen_code (temp) ; }

| '=' expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s =", $2.code, $3.code) ;
                $$ .code = gen_code (temp) ; }

| '<' expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s <", $2.code, $3.code) ;
                $$ .code = gen_code (temp) ; }

| '>' expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s >", $2.code, $3.code) ;
                $$ .code = gen_code (temp) ; }
```

```

| MET expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s <=", $2.code, $3.code) ;
    $$code = gen_code (temp) ; }

| GET expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s >=", $2.code, $3.code) ;
    $$code = gen_code (temp) ; }

| DISTINTO expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s = 0=", $2.code, $3.code) ;
    $$code = gen_code (temp) ; }

| AND expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s and", $2.code, $3.code) ;
    $$code = gen_code (temp) ; }

| OR expresion_parenthesis expresion_parenthesis { sprintf (temp, "%s %s or", $2.code, $3.code) ;
    $$code = gen_code (temp) ; }

| NOT expresion_parenthesis %prec UNARY_SIGN { sprintf (temp, "%s 0=", $2.code) ;
    $$code = gen_code (temp) ; }

| '(' '-' expresion_parenthesis ')' %prec UNARY_SIGN { sprintf (temp, "-%s", $3.code) ;
    $$code = gen_code (temp) ; }

;

```

// declaracion de operando

```

operando: IDENTIF { sprintf (temp, "%s @", $1.code) ;
    $$code = gen_code (temp) ; }

```

```
| NUMBER          { sprintf (temp, "%d", $1.value) ;  
                    $$code = gen_code (temp) ; }
```

```
;
```

```
%%                // SECCION 4 Codigo en C
```

```
int n_line = 1 ;
```

```
int yyerror (mensaje)
```

```
char *mensaje ;
```

```
{
```

```
    fprintf (stderr, "%s en la linea %d\n", mensaje, n_line) ;
```

```
    printf ( "\n" ) ; // bye
```

```
    return 0 ;
```

```
}
```

```
char *int_to_string (int n)
```

```
{  
    sprintf (temp, "%d", n) ;  
    return gen_code (temp) ;  
}
```

```
char *char_to_string (char c)  
{  
    sprintf (temp, "%c", c) ;  
    return gen_code (temp) ;  
}
```

```
char *my_malloc (int nbytes)    // reserva n bytes de memoria dinamica  
{  
    char *p ;  
    static long int nb = 0;    // sirven para contabilizar la memoria  
    static int nv = 0 ;        // solicitada en total  
  
    p = malloc (nbytes) ;  
    if (p == NULL) {
```

```
    fprintf(stderr, "No queda memoria para %d bytes mas\n", nbytes) ;  
    fprintf(stderr, "Reservados %ld bytes en %d llamadas\n", nb, nv) ;  
    exit (0) ;  
}  
nb += (long) nbytes ;  
nv++ ;  
  
return p ;  
}
```

```
/*  
***** Seccion de Palabras Reservadas *****  
*/
```

```
typedef struct s_keyword { // para las palabras reservadas de C  
    char *name ;  
    int token ;  
} t_keyword ;
```

```
t_keyword keywords [] = { // define las palabras reservadas y los --- Se ponen las llaves para evitar warnings de compilacion
```

```
    {"main",    MAIN},    // y los token asociados
```

```
    {"int",    INTEGER},
```

```
    {"setq",    SETQ},
```

```
    {"setf",    SETF},
```

```
    {"defun",    DEFUN},
```

```
    {"print",    PRINT},
```

```
    {"princ",    PRINC},
```

```
    {"loop",    LOOP},
```

```
    {"while",    WHILE},
```

```
    {"do",    DO},
```

```
    {"not",    NOT},
```

```
    {"and",    AND},
```

```
    {"or",    OR},
```

```
    {"mod",    MOD},
```

```
    {"<=",    MET},
```

```
    {">=",    GET},
```

```
    {"+",    '+'},    // para los operadores
```

```

{"-",    '-'},
{"*",    '*'},
{"/",    '/'},
{"/=",   DISTINTO},
{"if",    IF},
{"progn",  PROGN},
{"return-from",  RETURN_FROM},
{NULL,    0}      // para marcar el fin de la tabla
};

```

```

t_keyword *search_keyword (char *symbol_name)
{
    // Busca n_s en la tabla de pal. res.

    // y devuelve puntero a registro (simbolo)

    int i ;

    t_keyword *sim ;

    i = 0 ;

    sim = keywords ;

    while (sim [i].name != NULL) {

```



```

        if (strcmp (sim [i].name, symbol_name) == 0) {

                // strcmp(a, b) devuelve == 0 si a==b

        return &(sim [i]) ;

        }

        i++ ;

}

return NULL ;

}

```

```

/*****/

/***** Seccion del Analizador Lexicografico *****/

/*****/

```

```

char *gen_code (char *name)  // copia el argumento a un
{
        // string en memoria dinamica

        char *p ;

        int l ;

```

```
l = strlen (name)+1 ;  
p = (char *) my_malloc (l) ;  
strcpy (p, name) ;  
  
return p ;  
}
```

```
int yylex ()  
{  
// NO MODIFICAR ESTA FUNCION SIN PERMISO  
  
int i ;  
unsigned char c ;  
unsigned char cc ;  
char ops_expandibles [] = "!<=|>%&/+ -*" ;  
char temp_str [256] ;  
t_keyword *symbol ;
```

```
do {  
    c = getchar () ;  
  
    if (c == '#') { // Ignora las lineas que empiezan por # (#define, #include)  
        do {          //      OJO que puede funcionar mal si una linea contiene #  
            c = getchar () ;  
        } while (c != '\n') ;  
    }  
}
```

```
if (c == '/') { // Si la linea contiene un / puede ser inicio de comentario  
    cc = getchar () ;  
    if (cc != '/') { // Si el siguiente char es / es un comentario, pero...  
        ungetc (cc, stdin) ;  
    } else {  
        c = getchar () ; // ...  
        if (c == '@') { // Si es la secuencia //@ ==> transcribimos la linea  
            do {          // Se trata de codigo inline (Codigo embebido en C)  
                c = getchar () ;  
                putchar (c) ;  
            }
```

```
    } while (c != '\n');  
} else {      // ==> comentario, ignorar la linea  
    while (c != '\n') {  
        c = getchar ();  
    }  
}  
}  
} else if (c == '\\') c = getchar ();
```

```
if (c == '\n')  
    n_line++;
```

```
} while (c == ' ' || c == '\n' || c == 10 || c == 13 || c == '\t');
```

```
if (c == '\"') {  
    i = 0;  
    do {  
        c = getchar ();  
        temp_str [i++] = c;
```

```

} while (c != '\"' && i < 255);

if (i == 256) {

    printf ("AVISO: string con mas de 255 caracteres en linea %d\n", n_line);

}          // habria que leer hasta el siguiente " , pero, y si falta?

temp_str [--i] = '\0';

yyval.code = gen_code (temp_str);

return (STRING);

}

```

```

if (c == '.' || (c >= '0' && c <= '9')) {

    ungetc (c, stdin);

    scanf ("%d", &yyval.value);

//    printf ("\nDEV: NUMBER %d\n", yyval.value);    // PARA DEPURAR

    return NUMBER;

}

```

```

if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) {

    i = 0;

    while (((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) ||

```

```

    (c >= '0' && c <= '9') || c == '_') && i < 255){
        temp_str [i++] = tolower (c) ;
        c = getchar () ;
    }
    temp_str [i] = '\0' ;
    ungetc (c, stdin) ;

    yylval.code = gen_code (temp_str) ;
    symbol = search_keyword (yylval.code) ;
    if (symbol == NULL) { // no es palabra reservada -> identificador antes variable
//      printf ("\nDEV: IDENTIF %s\n", yylval.code) ; // PARA DEPURAR
        return (IDENTIF) ;
    } else {
//      printf ("\nDEV: OTRO %s\n", yylval.code) ; // PARA DEPURAR
        return (symbol->token) ;
    }
}

if (strchr (ops_expandibles, c) != NULL) { // busca c en ops_expandibles

```

```
cc = getchar () ;  
sprintf (temp_str, "%c%c", (char) c, (char) cc) ;  
symbol = search_keyword (temp_str) ;  
if (symbol == NULL) {  
    ungetc (cc, stdin) ;  
    yylval.code = NULL ;  
    return (c) ;  
} else {  
    yylval.code = gen_code (temp_str) ; // aunque no se use  
    return (symbol->token) ;  
}  
}  
  
// printf ("\nDEV: LITERAL %d #c#\n", (int) c, c) ;    // PARA DEPURAR  
if (c == EOF || c == 255 || c == 26) {  
//     printf ("tEOF ") ;                // PARA DEPURAR  
    return (0) ;  
}
```

```
    return c ;  
}
```

```
int main ()  
{  
    yyparse () ;  
}
```